



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NACC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Applied Machine Learning for Industry Solutions (PECO311C)
**Unit -3 : DESIGNING AND IMPLEMENTING ML SOLUTIONS WITH
MODERN TOOLS**

Dr. T.Bhaskar
Associate Professor

Google-Site: <https://sites.google.com/view/bhaskart/ug-pg-notes/applied-machine-learning-for-industry-solutions>

Moodle-Site: <https://proftbhaskar.gnomio.com/course/view.php?id=6> (Access as a Guest)



Contents

- **Introduction to ML Frameworks TensorFlow, PyTorch, and Scikit learn. Understanding their architecture, functionalities, and suitability for various industrial applications.**
- **Model Development and Training Hands-on experience in building, training, and validating machine learning models using real-world datasets. Emphasis on best practices for model development, including data preprocessing, feature engineering, and hyperparameter tuning.**
- **Deployment Strategies model serialization, Flask- RESTful APIs, containerization using Docker, and deployment on cloud platforms like AWS and Azure.**



Introduction to ML Frameworks TensorFlow, PyTorch, and Scikit learn

Introduction to ML Frameworks

1. **TensorFlow** Developed by Google.

Great for deep learning and neural networks.

Has both high-level APIs (Keras) and low-level APIs for flexibility.

Used in production environments and research.

2. **PyTorch** Developed by Facebook.

Popular for research and prototyping because of its dynamic computation graph.

Very intuitive for Python users.

Also supports deep learning and neural networks.

3. Scikit-learn Scikit-learn was initially developed by David Cournapeau as a Google Summer of Code project in 2007. Since then, scikit-learn has evolved into a widely used and community-driven open-source project with contributions from numerous volunteers and support from organizations like **Inria and NumFOCUS**. A classical ML library built on top of NumPy, SciPy, and matplotlib. Focused on traditional ML algorithms (regression, classification, clustering). Easy to use for beginners and useful for smaller projects.



TensorFlow is an **open-source machine learning (ML) and deep learning (DL) framework** developed by **Google Brain Team**.

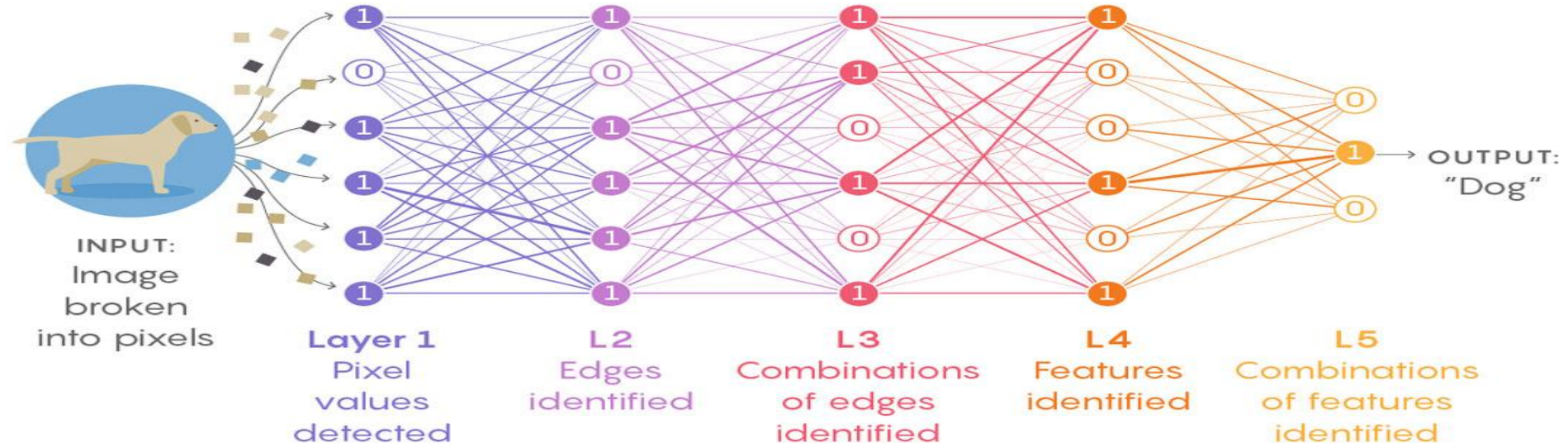
It is designed for:

- Building **neural networks**
- Handling **large-scale data**
- Running on **CPU, GPU, or TPU** for speed
- Deploying ML models on **mobile, web, or cloud**

Image classification :Sample Example for Deep Neural Networks

Learning From Experience

Deep neural networks learn by adjusting the strengths of their connections to better convey input signals through multiple layers to neurons associated with the right general concepts.



When data is fed into a network, each artificial neuron that fires (labeled "1") transmits signals to certain neurons in the next layer, which are likely to fire if multiple signals are received. The process filters out noise and retains only the most relevant features.



- CPU (Central Processing Unit):
 - General-purpose processor, “brain” of the computer.
 - Handles all logic, calculations, and input/output tasks.
 - Best for serial processing, light multitasking, and flexible software.
- GPU (Graphics Processing Unit):
 - Specialized for parallel computing, thousands of small cores.
 - Excels at massive parallel tasks: graphics rendering, deep learning, scientific calculation.
 - Dramatically speeds up model training for large datasets and image/video processing.
- TPU (Tensor Processing Unit):
 - Google-designed hardware purpose-built for machine learning and tensor operations.
 - Outstanding efficiency and speed for deep learning, especially using TensorFlow.
 - Designed for "matrix-heavy" model training and inference.



A **tensor** is basically a **multi-dimensional array** — the fundamental data structure used in frameworks like TensorFlow and PyTorch.

Think of it as a generalization of scalars, vectors, and matrices:

Rank	Name	Example Shape	Example
0	Scalar	()	7
1	Vector	(3,)	[1, 2, 3]
2	Matrix	(2, 3)	[[1,2,3], [4,5,6]]
3+	Tensor	(2, 3, 4)	A batch of 2 matrices, each with shape (3, 4)



Tensors

A tensor is a multi-dimensional array.

Rank	Name	Example Shape	Example
0	Scalar	()	7
1	Vector	(3,)	[1, 2, 3]
2	Matrix	(2, 3)	[1, 2, 3] 4, 5, 6
3+	Tensor	(2, 3, 4) [1, 2, 3, 5] 12	[1, 2, 3, 4] 5, 6, 7, 8 9, 10, 11, 12

Examples



Image



Video



Text
embeddings



Why Tensors Are Important

- In **deep learning**, all data (images, audio, text, etc.) is converted into tensors before training.
- GPUs process tensors very efficiently using **parallel computation**.
- Tensor operations (addition, multiplication, reshaping, slicing) are highly optimized.

```
import tensorflow as tf
```

```
# Scalar
```

```
scalar = tf.constant(7)  
print(scalar.shape) # ()
```

```
# Vector
```

```
vector = tf.constant([1, 2, 3])  
print(vector.shape) # (3,)
```

```
# Matrix
```

```
matrix = tf.constant([[1, 2], [3, 4]])  
print(matrix.shape) # (2, 2)
```

```
# 3D Tensor
```

```
tensor_3d = tf.constant([[[1, 2], [3, 4]],  
                        [[5, 6], [7, 8]]])  
print(tensor_3d.shape) # (2, 2, 2)
```



Speed Comparison (Typical Numbers)

- CPU: Slowest for deep learning workloads; best for small models, irregular computations, flexible tasks.
- GPU: Up to 10-100 times faster than CPUs for parallel workloads.
- TPU: Can be 15-30 times faster than top contemporary GPUs and CPUs for AI tasks, especially neural network inference.

The latest TPU chips (e.g., Trillium) are 4.7x faster than previous TPUs, using less power and handling larger models.

Hardware	Best For	Speed	Typical Use Cases	Flexibility
CPU	Versatile, small-scale tasks	Slow for ML/AI	OS operations, basic ML, prototyping	Highest
GPU	Parallel, large-scale ML	Much faster than CPU	Deep learning, graphics, scientific analysis	High
TPU	Model training/inference (ML)	Up to 30x faster than GPU/CPU	Neural network training, TensorFlow models	Low (TensorFlow focused)



TensorFlow Architecture

TensorFlow's architecture consists of **three main parts**:

1. Preprocessing Data

1. Load and prepare datasets.
2. Example: Normalization, tokenization, image resizing.

2. Building Models

1. Define layers and network structure.
2. Can be built using:
 1. **Low-level API** (manually defining computation graph)
 2. **Keras High-level API** (easy & beginner-friendly)

3. Training & Deployment

1. Train models using gradient descent.
2. Deploy models to:
 1. Mobile (TensorFlow Lite)
 2. Web (TensorFlow.js)
 3. Edge devices (TensorFlow Lite Micro)



Functionalities of TensorFlow

- **Deep Learning** (CNN, RNN, Transformers, etc.)
- **Machine Learning** (regression, classification, clustering)
- **Computer Vision** (image classification, object detection)
- **Natural Language Processing** (text classification, translation)
- **Reinforcement Learning**
- **Model Serving** (TensorFlow Serving for APIs)



Suitability for Industrial Applications

TensorFlow is widely used in industries like:

- **Healthcare** → Disease detection, medical imaging
- **Finance** → Fraud detection, stock prediction
- **Manufacturing** → Predictive maintenance, defect detection
- **Retail** → Product recommendation, inventory forecasting
- **Autonomous Vehicles** → Object detection, navigation



TensorFlow: Simple Neural Network for MNIST Digit Classification

Used MNIST dataset: 28x28 grayscale images of handwritten digits (0-9).

Preprocess data: scale pixel values and one-hot encode labels.

Build a simple neural network with one hidden layer.

Compile the model (specify optimizer, loss, metrics).

Train the model on training data.

Evaluate performance on test data.



```
# Install TensorFlow (usually pre-installed in Colab)
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical

# Load data
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values (0-255 to 0-1)
x_train, x_test = x_train / 255.0, x_test / 255.0

# One-hot encode target labels
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```



```
# Build model
```

```
model = Sequential([  
    Flatten(input_shape=(28,28)),  
    Dense(128, activation='relu'),  
    Dense(10, activation='softmax')  
])
```

```
# Compile model
```

```
model.compile(optimizer='adam',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

```
# Train model
```

```
model.fit(x_train, y_train, epochs=5, validation_split=0.2)
```

```
# Evaluate model
```

```
test_loss, test_acc = model.evaluate(x_test, y_test)  
print(f"Test accuracy: {test_acc:.4f}")
```



Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— **0s** 0us/step

/usr/local/lib/python3.11/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape`/
super().__init__(**kwargs)

Epoch 1/5

1500/1500 ————— **16s** 9ms/step - accuracy: 0.8635 - loss: 0.4772 - val_accuracy: 0.9538 - val_loss: 0.1584

Epoch 2/5

1500/1500 ————— **20s** 9ms/step - accuracy: 0.9602 - loss: 0.1364 - val_accuracy: 0.9623 - val_loss: 0.1195

Epoch 3/5

1500/1500 ————— **14s** 5ms/step - accuracy: 0.9746 - loss: 0.0865 - val_accuracy: 0.9707 - val_loss: 0.0999

Epoch 4/5

1500/1500 ————— **6s** 4ms/step - accuracy: 0.9811 - loss: 0.0622 - val_accuracy: 0.9701 - val_loss: 0.1034

Epoch 5/5

1500/1500 ————— **9s** 6ms/step - accuracy: 0.9864 - loss: 0.0505 - val_accuracy: 0.9743 - val_loss: 0.0878

313/313 ————— **1s** 2ms/step - accuracy: 0.9715 - loss: 0.0923

Test accuracy: 0.9761



PyTorch

PyTorch is a widely used, open-source deep learning framework, praised for its ease-of-use, flexibility (dynamic computation graphs), and extensive ecosystem. Originally developed by Meta(Facebook), PyTorch is the first choice for research and practical applications in computer vision, NLP, and more.

◇ Architecture:

- Dynamic Computation Graph:** More flexible, changes on the fly (good for research).
- Autograd Engine:** Automatic differentiation for backpropagation.
- TorchScript:** For exporting models to production.

◇ Functionalities:

- State-of-the-art deep learning models
- Easy debugging (Pythonic style)
- GPU acceleration
- Hugging Face Transformers integration

◇ Industry Use Cases:

- Facebook AI uses PyTorch for computer vision & NLP research.
- Autonomous driving research (Tesla, Uber)
- Drug discovery (pharmaceutical AI models)



PyTorch: Simple Neural Network for MNIST Digit Classification

Download and load MNIST dataset with normalization.

Define a simple neural network class with two layers.

Define loss function (CrossEntropy) and optimizer (Adam).

Write training loop over epochs and batches.

Evaluate model accuracy on test data.



PyTorch: Simple Neural Network for MNIST Digit Classification

```
# Install PyTorch (usually pre-installed in Colab)
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Define transformations to normalize data
transform = transforms.Compose([transforms.ToTensor(),
                                transforms.Normalize((0.1307,), (0.3081,))])

# Download and load datasets
train_dataset = datasets.MNIST(root='./data', train=True, download=True,
                                transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True,
                                transform=transform)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)
```



PyTorch: Simple Neural Network for MNIST Digit Classification

```
# Define simple neural network
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(28*28, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 28*28)  # Flatten input
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = Net()

# Loss and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```



PyTorch: Simple Neural Network for MNIST Digit Classification

```
# Training loop
for epoch in range(5):
    model.train()
    for data, target in train_loader:
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output,
target)
        loss.backward()
        optimizer.step()
    print(f"Epoch {epoch+1}
completed")
```

```
# Testing
model.eval()
correct = 0
total = 0
with torch.no_grad():
    for data, target in test_loader:
        output = model(data)
        _, predicted = torch.max(output, 1)
        total += target.size(0)
        correct += (predicted == target).sum().item()

print(f"Test Accuracy: {100 * correct / total:.2f}%")
```



PyTorch: Simple Neural Network for MNIST Digit Classification

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 138MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 26.1MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 78.6MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 4.84MB/s]
Epoch 1 completed
Epoch 2 completed
Epoch 3 completed
Epoch 4 completed
Epoch 5 completed
Test Accuracy: 97.47%
```



Scikit-learn

- Scikit-learn is a widely used Python machine learning library focused on classic algorithms (regression, classification, clustering, etc.), with a simple API and broad compatibility.
- By default, Scikit-learn is CPU-based. All algorithms run efficiently on standard CPUs, making it ideal for prototyping, learning, and moderate-size data model training.

◇ Architecture:

- Estimator API:** For training models.
- Transformer API:** For preprocessing.
- Pipeline:** Combines preprocessing + modeling.

◇ Functionalities:

- Classical ML algorithms (Logistic Regression, Random Forest, Naive Bayes, SVM, etc.)
- Feature scaling & transformation
- Model evaluation (accuracy, precision, recall, ROC)
- Works well for small-to-medium datasets

◇ Industry Use Cases:

- Credit scoring & fraud detection
- Predictive maintenance in manufacturing
- Customer segmentation in marketing
- Forecasting sales or demand



Scikit-learn: MNIST Digit Classification using Logistic Regression

Dataset – `load_digits()` loads a small set of handwritten digit images (8×8 pixels).

`digits.images` shape: (1797, 8, 8); `digits.data` shape: (1797, 64) (each image flattened to 64 features).

Visualization – Plot the first 8 images with their true labels so you can see what the inputs look like.

Train/Test Split – Split the data into training and testing parts (commonly 80% train, 20% test) using a fixed `random_state` for reproducibility.

(Optional) Feature scaling – Pixel values are small (0–16) here so scaling is usually optional; for other datasets use `StandardScaler` or `MinMaxScaler`.

Logistic Regression Model – Create a multinomial logistic regression model.

It learns weight vectors that map the 64 input features to 10 output classes.

Uses softmax to convert scores to class probabilities.

Training – `model.fit(X_train, y_train)` runs an optimizer (e.g., `lbfgs`) to minimize cross-entropy loss and find the best weights.

Use `max_iter` large enough (e.g., 1000) to ensure convergence.

Prediction – `model.predict(X_test)` computes class scores for each test sample and picks the class with the highest probability (`argmax`).

Evaluation – Measure performance with accuracy and a classification report showing precision, recall, and F1-score per digit.

Optionally plot a confusion matrix to see which digits are commonly confused.



```
# STEP 1: Import required libraries
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# STEP 2: Load digits dataset
digits = load_digits()
X, y = digits.data, digits.target

# STEP 3: Visualize some images
plt.figure(figsize=(6, 4))
for i in range(8):
    plt.subplot(2, 4, i + 1)
    plt.imshow(digits.images[i], cmap='gray')
    plt.title(f"Label: {digits.target[i]}")
    plt.axis('off')
plt.show()
```

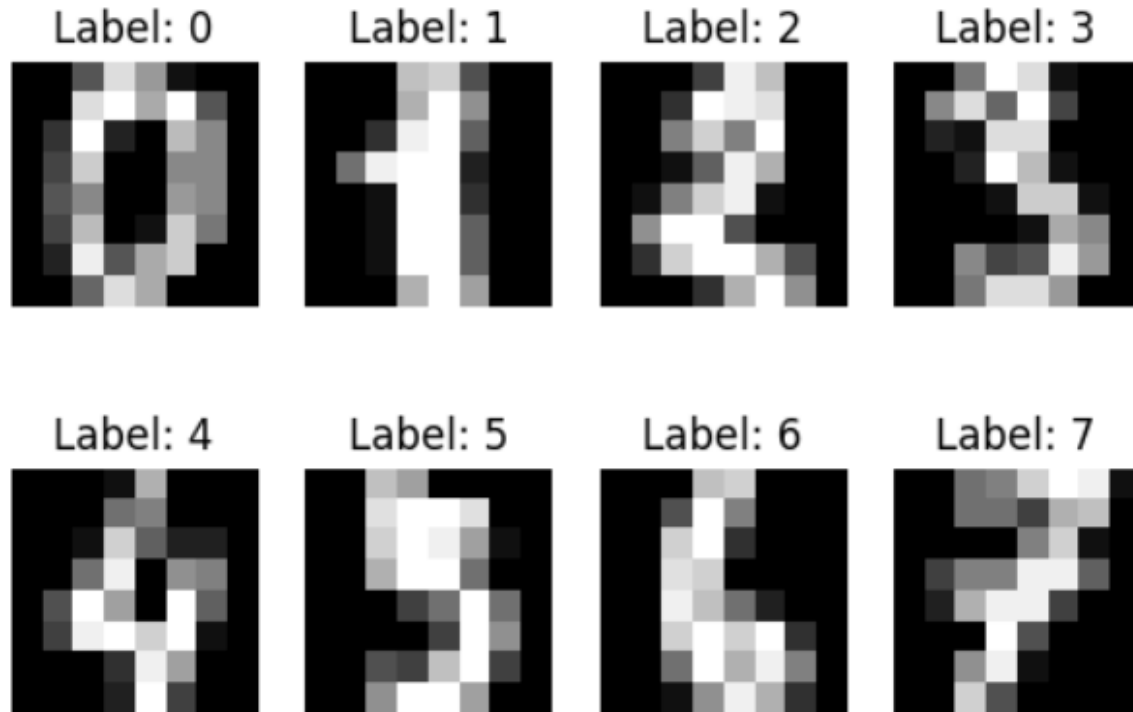


```
# STEP 4: Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# STEP 5: Create and train Logistic Regression model
# max_iter increased to ensure convergence
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# STEP 6: Predict on test set
y_pred = model.predict(X_test)

# STEP 7: Evaluate performance
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n",
classification_report(y_test, y_pred))
```



Accuracy: 0.975

Classification Report:				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	33
1	0.97	1.00	0.98	28
2	1.00	1.00	1.00	33
3	0.97	0.97	0.97	34
4	1.00	0.98	0.99	46
5	0.92	0.96	0.94	47
6	0.97	0.97	0.97	35
7	1.00	0.97	0.99	34
8	0.97	0.97	0.97	30
9	0.97	0.95	0.96	40
accuracy			0.97	360
macro avg	0.98	0.98	0.98	360
weighted avg	0.98	0.97	0.98	360



Summary

Framework	Use case	Strengths	Beginner Friendliness
TensorFlow	Deep Learning (large scale)	Production ready, Keras API	Moderate (high-level API helps)
PyTorch	Research & Prototyping	Dynamic graph, intuitive Pythonic	Easy to learn for Python users
Scikit-learn	Classical ML algorithms	Simple API, lots of ML tools	Easiest for traditional ML



Hands-On Module: Model Development & Training Learning Objectives

- Load and explore a real-world dataset.
 - Apply data preprocessing techniques.
 - Perform feature engineering.
 - Build, train, and validate a machine learning model.
 - Perform hyperparameter tuning for optimal performance.
-
- ☑ **Data Preprocessing** – Handling missing values, scaling, encoding
 - ☑ **Feature Engineering** – Encoding categorical features
 - ☑ **Model Building** – RandomForest as a base example
 - ☑ **Validation** – Train/test split & metrics
 - ☑ **Hyperparameter Tuning** – Grid search with cross-validation



Dataset Selection

use the **Titanic survival dataset** (`seaborn.load_dataset("titanic")` or Kaggle version) because:

- It's real-world.
- It has both categorical & numerical features.
- It allows clear preprocessing examples.

```
import pandas as pd
import seaborn as sns

# Load dataset
df = sns.load_dataset("titanic")
print(df.head())
print(df.info())
```



RangeIndex: 891 entries, 0 to 890

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	714 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64
7	embarked	889 non-null	object
8	class	891 non-null	category
9	who	891 non-null	object
10	adult_male	891 non-null	bool
11	deck	203 non-null	category
12	embark_town	889 non-null	object
13	alive	891 non-null	object
14	alone	891 non-null	bool

dtypes: bool(2), category(2), float64(2), int64(4), object(5)

memory usage: 80.7+ KB

None



```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

# Separate features and target
X = df.drop("survived", axis=1)
y = df["survived"]

# Select numerical and categorical columns
num_features = X.select_dtypes(include=["int64", "float64"]).columns
cat_features = X.select_dtypes(include=["object", "category"]).columns
```



```
# Preprocessing pipeline
num_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

cat_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("num", num_transformer, num_features),
    ("cat", cat_transformer, cat_features)
])

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)
```

Data Preprocessing

- Handle missing values
- Encode categorical variables
- Scale numerical variables



Training & Validation

```
from sklearn.metrics import accuracy_score, classification_report

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Create pipeline
clf = Pipeline(steps=[("preprocessor", preprocessor),
                      ("model", model)])

# Train model
clf.fit(X_train, y_train)

# Validate
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Accuracy: 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	105
1	1.00	1.00	1.00	74
accuracy			1.00	179
macro avg	1.00	1.00	1.00	179
weighted avg	1.00	1.00	1.00	179



Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "model__n_estimators": [50, 100, 200],
    "model__max_depth": [None, 5, 10]
}

grid_search = GridSearchCV(clf, param_grid, cv=5,
scoring="accuracy")
grid_search.fit(X_train, y_train)

print("Best parameters:", grid_search.best_params_)
print("Best accuracy:", grid_search.best_score_)

Best parameters: {'model__max_depth': None, 'model__n_estimators': 50}
Best accuracy: 1.0
```



Hyperparameter Tuning in Machine Learning

Hyperparameter tuning is the process of selecting the optimal values for configuration settings—known as hyperparameters—that are set before training a machine learning model. These values impact how the model learns and generalizes, affecting accuracy, efficiency, and the risk of over- or underfitting

Common Hyperparameters

- Learning rate: Speed at which the model updates weights
- Number of layers/nodes: In neural networks
- Batch size: Number of samples per training iteration
- Epochs: Times the dataset is passed through the model
- Regularization strength: Controls model complexity
- Tree depth / number in forests: For tree-based models



Hyperparameter Tuning in Machine Learning

Popular Tuning Techniques

1. Grid Search

- Tests all possible combinations from user-specified ranges.
- Finds the best parameter set by exhaustive search.
- Computationally expensive, can be slow for many hyperparameters.

2. Random Search

- Samples random combinations from the search space.
- Often faster and surprisingly effective, especially for high-dimensional spaces.

3. Bayesian Optimization

- Builds a probabilistic model of the function mapping hyperparameters to performance.
- Selects new hyperparameter sets based on previous results (explores efficiently).
- Often finds optimal settings with fewer trials than grid or random search.



Hyperparameter Tuning in Machine Learning

Other Techniques

- **Manual tuning:** Tweaking values based on expertise and iterative feedback.
- **Cross-validation:** Ensures that each hyperparameter configuration is validated on multiple data splits, improving robustness.
- **Early stopping:** Halts training when improvement plateaus, useful in deep learning.
- **Automated tools:** Libraries like HyperOpt, Optuna, and Ray Tune offer advanced tuning strategies and support for distributed search.

Advantages

- Increases model accuracy and generalization
- Prevents underfitting and overfitting
- Improves efficiency, often reducing training time
- Leads to tailored models suited for specific data and tasks



Hyperparameter Tuning in Machine Learning

Leading Tools for Hyperparameter Optimization

Tool	Method(s)	Highlights
GridSearchCV	Grid, Random	Simple, easy for smaller problems
HyperOpt	Random, Bayesian (TPE)	Handles complex spaces, scalable
Optuna	Bayesian, Random	Efficient, pruning and distributed
Ray Tune	Many, incl. PBT	Distributed, advanced algorithms



Deployment Strategies for Machine Learning Models

Model Serialization

When we train a Machine Learning model, we need to **save** it so it can be reused for predictions without retraining.

•Why Serialization?

- Save time → no need to retrain the model every time.
- Portability → use the trained model in production.
- Sharing → transfer models across teams.

•Common Formats:

- Pickle (.pkl)**: Python's default serialization library.
- Joblib (.joblib)**: Efficient for large NumPy arrays (preferred in ML).
- ONNX (.onnx)**: Cross-platform model format.

Example (using Joblib):

```
import joblib
joblib.dump(model, "model.pkl") # Save model
model = joblib.load("model.pkl") # Load model
```



Flask RESTful APIs for Model Serving

Example:

```
python

from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)
model = joblib.load("model.pkl")

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    prediction = model.predict([data["features"]])
    return jsonify({"prediction": prediction.tolist()})

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5000)
```

Flask helps expose the ML model as a **web service**.

•Steps:

- 1.Install Flask → pip install flask
- 2.Load the serialized model
- 3.Define an API endpoint (/predict)
- 4.Accept input data (JSON) and return predictions

Testing:

Use **Postman** or **curl**:

```
curl -X POST http://127.0.0.1:5000/predict -H
"Content-Type: application/json" -d
 '{"features":[5.1,3.5,1.4,0.2]}'
```



Containerization with Docker

Docker ensures the ML model runs **consistently** across different environments.

- **Steps:**

1. Install Docker
2. Create a **Dockerfile**

```
dockerfile
```

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

3. Build the image

```
bash
```

```
docker build -t ml-flask-app .
```

4. Run the container

```
bash
```

```
docker run -p 5000:5000 ml-flask-app
```

Advantages:

- Portability (works on any OS/cloud)
- Isolation (dependencies don't clash)
- Scalability (works with Kubernetes, ECS, AKS)



Deployment on Cloud Platforms

A. AWS Deployment

Options:

1.AWS EC2

- Launch EC2 instance
- Install Docker
- Run container → exposed via public IP

2.AWS Elastic Beanstalk

- Upload Flask app with [Dockerrun.aws.json](#)
- Auto handles scaling & load balancing

3.AWS SageMaker

- Directly deploy ML models
- Managed endpoints for inference

B. Azure Deployment

Options:

1.Azure App Service

1. Deploy Flask API directly from GitHub/DockerHub

2.Azure Container Instances (ACI)

1. Run Dockerized ML model without managing servers

3.Azure Kubernetes Service (AKS)

1. Large-scale deployments with orchestration



Comparison of Deployment Strategies

Strategy	Pros	Cons
Flask API	Easy to implement, lightweight	Not scalable alone
Docker	Portable, consistent environment	Needs Docker knowledge
AWS EC2	Flexible, full control	Manual scaling required
AWS SageMaker	ML-native, auto-scaling	Costly for small projects
Azure AppSvc	Simple web deployment	Limited for heavy ML workloads
AKS / ECS	High scalability, orchestration	Complex setup

- **Serialize** models → save & reuse.
- **Flask REST API** → make predictions accessible via HTTP.
- **Docker** → portable & reproducible deployments.
- **Cloud (AWS/Azure)** → scalable real-world deployments.



Real-time Fraud Detection System (Model Deployment & Efficiency) - Build and deploy a fraud detection model using logistic regression or XGBoost. Implement the model as a RESTful API and evaluate its inference time and memory usage under different loads.

Installs all the Python packages we need like FastAPI (for API), XGBoost & scikit-learn (machine learning), pandas (data handling), matplotlib (plotting), and others.
The -q means "quiet" install (less output).

```
!pip install -q fastapi uvicorn nest-asyncio pyngrok xgboost scikit-learn  
pandas matplotlib requests joblib gdown
```



Imports all the libraries used in the program.

```
# nest_asyncio allows running asynchronous code (like FastAPI) inside environments like Jupyter notebooks
import nest_asyncio

# pyngrok is used to create secure tunnels from your local server (like FastAPI) to the internet
from pyngrok import ngrok

# pandas is a powerful data manipulation library; used for loading, analyzing, and transforming data
import pandas as pd

# numpy is a numerical library used for working with arrays, matrices, and mathematical operations
import numpy as np

# train_test_split splits your dataset into training and testing sets for model evaluation
from sklearn.model_selection import train_test_split

# LogisticRegression is a simple machine learning model used for classification tasks
from sklearn.linear_model import LogisticRegression

# XGBClassifier is an efficient, high-performance gradient boosting classifier from the XGBoost library
from xgboost import XGBClassifier

# StandardScaler standardizes (scales) features so they have mean 0 and variance 1 – improves model
performance
from sklearn.preprocessing import StandardScaler
```



Imports all the libraries used in the program.

accuracy_score measures how often the model predicts the correct class — used for evaluating classification models
from sklearn.metrics import accuracy_score

matplotlib is used for creating visualizations like plots and charts
import matplotlib.pyplot as plt

FastAPI is a modern web framework used to build APIs quickly and efficiently
from fastapi import FastAPI, Request

FastAPI response classes: return data in different formats like JSON, files, or HTML
from fastapi.responses import JSONResponse, FileResponse, HTMLResponse

uvicorn is a web server used to run FastAPI applications
import uvicorn

joblib is used to save and load trained machine learning models efficiently
import joblib



Imports all the libraries used in the program.

requests allows your program to make HTTP requests (e.g., fetch data from APIs or websites)
`import requests`

BytesIO lets you read/write binary data in memory — useful when dealing with images or files over the web
`from io import BytesIO`

PIL (Python Imaging Library) is used for opening, manipulating, and saving image files
`from PIL import Image`

display and Markdown are used in Jupyter notebooks to show rich output (formatted text, images, etc.)
`from IPython.display import display, Markdown`

threading allows running multiple operations in parallel (multithreading)
`import threading`

time module helps with operations like delays, measuring execution time, and working with timestamps
`import time`

gdown allows you to download files directly from Google Drive programmatically
`import gdown`

tracemalloc is used for tracking memory usage and detecting memory leaks in your Python program
`import tracemalloc`

ThreadPoolExecutor is used to run functions in parallel threads; `as_completed` helps manage the results
`from concurrent.futures import ThreadPoolExecutor, as_completed`



Download and load the Credit Card Fraud Detection dataset from Kaggle: I have download a real credit card fraud dataset from Google Drive using **gdown**.
Save it as a CSV file.
Load the CSV file into a pandas DataFrame df.
Print dataset shape (rows, columns) and first few rows to check data.

```
# ----- Step 1: Download and load real dataset --  
-----  
file_id = "1nmtDEUSWJvYlv6441QsFI4m-EgsLyG34"  
download_url =  
f"https://drive.google.com/uc?id={file_id}"  
csv_filename = "5.creditcard.csv"  
  
print("Downloading dataset from Google Drive...")  
gdown.download(download_url, csv_filename,  
quiet=False)  
  
df = pd.read_csv(csv_filename)  
print("Dataset shape:", df.shape)  
print(df.head())
```



Downloading...

From (original): <https://drive.google.com/uc?id=1nmtDEUSWJvYlv6441QsFI4m-EgsLyG34>

From (redirected): <https://drive.google.com/uc?id=1nmtDEUSWJvYlv6441QsFI4m-EgsLyG34&confirm=t&uuid=5540232b-4022-4eb5-b3f8-76d745645deb>

To: /content/5.creditcard.csv

100%|██████████| 151M/151M [00:01<00:00, 79.7MB/s]

Dataset shape: (284807, 31)

	Time	V1	V2	V3	V4	V5	V6	V7	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	

	V8	V9	...	V21	V22	V23	V24	V25	\
0	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	
1	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	
2	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	
3	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	
4	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	

	V26	V27	V28	Amount	Class
0	-0.189115	0.133558	-0.021053	149.62	0
1	0.125895	-0.008983	0.014724	2.69	0
2	-0.139097	-0.055353	-0.059752	378.66	0
3	-0.221929	0.062723	0.061458	123.50	0
4	0.502292	0.219422	0.215153	69.99	0

[5 rows x 31 columns]



Select features and target The target column is the label we want to predict (fraud or not). Usually named 'Class' here.

Separate features (X_all) and labels (y).

Select only the first 10 numeric features (because we want 10 inputs).

Save feature names to feature_names for later use.

Extract fraud labels (target) and pick 10 numeric features as inputs for models.

```
# Determine target column
target_column = 'Class' if 'Class' in df.columns else 'fraud'
if target_column not in df.columns:
    raise ValueError(f"Target column '{target_column}' not found. Columns: {df.columns}")

# Select numeric features only, take first 10
X_all = df.drop(columns=[target_column])
y = df[target_column]

X_numeric = X_all.select_dtypes(include=np.number).iloc[:, :10]
feature_names = X_numeric.columns.tolist()
print(f"Using features ({len(feature_names)}): {feature_names}")
print(df.columns.tolist())
```

```
➡ Using features (10): ['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9']
['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16',
```

Scale data and split into train/test Features are scaled to zero mean and unit variance using StandardScaler (improves ML training).

Split data into training (70%) and testing (30%) sets, maintaining class distribution with stratify=y. StandardScaler → makes each feature mean=0, variance=1 → helps models converge faster.

train_test_split:

test_size=0.3 → 30% test, 70% train.

random_state=42 → reproducibility.

stratify=y → keep fraud/non-fraud ratio same.

Scale features and split dataset into train/test while preserving fraud ratio.

```
▶ scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_numeric)  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.3, random_state=42, stratify=y  
)  
print(X_scaled)
```

```
⇒ [[-1.99658302 -0.69424232 -0.04407492 ... 0.19367894 0.08263728  
    0.33112778]  
   [-1.99658302 0.60849633 0.16117592 ... -0.06370021 0.07125348  
   -0.23249419]  
   [-1.99656197 -0.69350046 -0.81157783 ... 0.63977564 0.20737273  
   -1.37867535]]
```



Train two ML models :Train Logistic Regression and XGBoost classifiers on training data.

Predict on test data.

Calculate and print accuracy for both models. Train Logistic Regression & XGBoost on training data.

Predict & calculate accuracy.

Save models & scaler for deployment using joblib.dump.

Train Logistic Regression & XGBoost, evaluate accuracies. This creates an instance of the LogisticRegression model from scikit-learn.

max_iter=1000: Sets the maximum number of iterations for the solver to converge.

Sometimes Logistic Regression requires more iterations to find the optimal solution, especially on larger or complex datasets.

Uses the trained logistic regression model to predict labels for the test data X_test.

The predictions are stored in the variable log_pred.

Creates an instance of the XGBoost classifier from the xgboost library.

use_label_encoder=False: Disables label encoding that XGBoost used by default (this is to avoid warnings).

eval_metric='logloss': Sets the evaluation metric to log loss, which is suitable for binary classification problems.

Prints the accuracy scores for both models.

{log_acc:.4f} means the accuracy will be formatted to 4 decimal places.



Train two ML models :Train Logistic Regression and XGBoost classifiers on training data.
Predict on test data.
Calculate and print accuracy for both models.

```
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train)
log_pred = log_reg.predict(X_test)
log_acc = accuracy_score(y_test, log_pred)

xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
xgb.fit(X_train, y_train)
xgb_pred = xgb.predict(X_test)
xgb_acc = accuracy_score(y_test, xgb_pred)

print(f"Logistic Regression Accuracy: {log_acc:.4f}")
print(f"XGBoost Accuracy: {xgb_acc:.4f}")
```

```
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [09:15:44] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
Logistic Regression Accuracy: 0.9988
XGBoost Accuracy: 0.9991
```



Save trained models and scaler : Save the trained models and scaler to files for loading later in API.

```
[ ] joblib.dump(log_reg, "log_reg_model.pkl")
    joblib.dump(xgb, "xgb_model.pkl")
    joblib.dump(scaler, "scaler.pkl")
```

➡ ['scaler.pkl']

Plot and save accuracy comparison graph

Create a bar graph to visually compare model accuracies.

Save the plot as model_comparison.png.

```
▶ plt.figure(figsize=(6,4))
  plt.bar(["Logistic Regression", "XGBoost"], [log_acc, xgb_acc], color=['blue', 'green'])
  plt.ylabel("Accuracy")
  plt.title("Model Accuracy Comparison")
  plt.savefig("model_comparison.png")
  plt.close()
```



Create FastAPI app with routes :Homepage: HTML form to input features and select model.
Shows input boxes for each feature and model dropdown.
Links to graphs.

```
@app.get("/", response_class=HTMLResponse)
async def homepage():
    form_html = """
    <h2>Fraud Detection Prediction</h2>
    <form action="/predict" method="post">
    """ + "".join(
        [f'{feat}: <input type="number" step="0.0001" name="{feat}" required><br>' for feat in
        feature_names]
    ) + """
    <br>
    Select Model:
    <select name="model">
        <option value="log_reg">Logistic Regression</option>
        <option value="xgb">XGBoost</option>
    </select><br><br>
    <input type="submit" value="Predict">
    </form>
    <br>
    <a href="/comparison_graph">View Model Accuracy Comparison Graph</a><br>
    <a href="/performance_graph">View Inference Time & Memory Usage Comparison Graph</a>
    """
    return HTMLResponse(content=form_html)
```



Prediction endpoint: Takes input features and selected model, predicts fraud or not.

Reads form data, converts inputs to floats.

Loads scaler and scales input.

Loads selected model and makes prediction.

Returns JSON with result.

/predict endpoint → handles POST request from form.

Reads user inputs, converts to floats.

Loads scaler → scales inputs.

Loads selected model (log_reg or xgb).

Runs prediction:

1 → Fraudulent Transaction.

0 → Legitimate Transaction.

Returns result in JSON format.

Add /predict API endpoint that takes inputs, scales them, loads model, and returns prediction in JSON.



Prediction endpoint: Takes input features and selected model, predicts fraud or not.

Reads form data, converts inputs to floats.

Loads scaler and scales input. Loads selected model and makes prediction. Returns JSON with result.

```
@app.post("/predict", response_class=JSONResponse)
async def predict(request: Request):
    form = await request.form()
    try:
        features = [float(form[feat]) for feat in feature_names]
    except Exception as e:
        return JSONResponse({"error": f"Invalid input: {e}"})

    model_name = form.get("model", "log_reg")
    scaler = joblib.load("scaler.pkl")
    features_scaled = scaler.transform([features])

    if model_name == "log_reg":
        model = joblib.load("log_reg_model.pkl")
    else:
        model = joblib.load("xgb_model.pkl")

    prediction = model.predict(features_scaled)[0]
    result = "Fraudulent Transaction" if prediction == 1 else "Legitimate
Transaction"

    return {"prediction": result, "model_used": model_name}
```



Graph serving endpoints:

Serve the saved accuracy and performance graphs as images.

```
[ ] @app.get("/comparison_graph")
    async def comparison_graph():
        return FileResponse("model_comparison.png")

    @app.get("/performance_graph")
    async def performance_graph():
        return FileResponse("performance_comparison.png")
```

Performance test - Measure inference time and memory usage under load Helper function to measure single inference time:

```
[ ] def measure_inference_time(model, features_scaled):
    start_time = time.time()
    _ = model.predict(features_scaled)
    return time.time() - start_time
```



Function to simulate many inference requests concurrently and measure average time & memory:

Uses Python's tracemalloc to track memory usage during concurrent predictions.

Runs num_requests predictions with concurrency threads.

Returns average inference time and memory increase during load.

num_requests=100 → simulate 100 requests.

concurrency=10 → run 10 at the same time (parallel).

tracemalloc → measures memory usage.

Returns average inference time and memory increase.

Load test models by simulating many requests and measure average inference time + memory increase.



Function to simulate many inference requests concurrently and measure average time & memory:

Uses Python's tracemalloc to track memory usage during concurrent predictions.

Runs num_requests predictions with concurrency threads. Returns average inference time and memory increase during load.

```
def load_test_model(model, features_scaled, num_requests=100,
concurrency=10):
    times = []

    tracemalloc.start()
    baseline_mem = tracemalloc.get_traced_memory()[0] / (1024 ** 2) # baseline
memory in MB

    def task():
        t = measure_inference_time(model, features_scaled)
        return t

    with ThreadPoolExecutor(max_workers=concurrency) as executor:
        futures = [executor.submit(task) for _ in range(num_requests)]
        for future in as_completed(futures):
            times.append(future.result())

    current_mem = tracemalloc.get_traced_memory()[1] / (1024 ** 2) # max
memory used in MB
    tracemalloc.stop()

    avg_time = sum(times) / len(times)
    mem_increase = current_mem - baseline_mem
    return avg_time, mem_increase
```



Run tests for both models:

```
[ ] sample_feature_scaled = X_test[0].reshape(1, -1)

avg_logreg_time, mem_logreg = load_test_model(log_reg, sample_feature_scaled)
avg_xgb_time, mem_xgb = load_test_model(xgb, sample_feature_scaled)
```



Print results:

```
print(f"Logistic Regression - Avg inference time: {avg_logreg_time*1000:.3f} ms, Memory increase: {mem_logreg:.4f} MB")
print(f"XGBoost - Avg inference time: {avg_xgb_time*1000:.3f} ms, Memory increase: {mem_xgb:.4f} MB")
```



```
Logistic Regression - Avg inference time: 4.089 ms, Memory increase: 0.2420 MB
XGBoost - Avg inference time: 14.888 ms, Memory increase: 0.4537 MB
```



Plot performance comparison graph

Plots two bar charts side by side: Average inference time in milliseconds.

Memory usage increase in MB during load test. Saves plot as performance_comparison.png.

```
plt.figure(figsize=(12,5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.bar(['Logistic Regression', 'XGBoost'], [avg_logreg_time*1000,  
avg_xgb_time*1000], color=['blue', 'green'])
```

```
plt.ylabel('Average Inference Time (ms)')
```

```
plt.title('Inference Time Comparison')
```

```
plt.subplot(1, 2, 2)
```

```
plt.bar(['Logistic Regression', 'XGBoost'], [mem_logreg, mem_xgb],  
color=['blue', 'green'])
```

```
plt.ylabel('Memory Usage Increase (MB)')
```

```
plt.title('Memory Usage Increase During Load')
```

```
plt.tight_layout()
```

```
plt.savefig("performance_comparison.png")
```

```
plt.close()
```



Run FastAPI with ngrok for public access

Sets ngrok token to create a public URL for your local API server.

Runs FastAPI using uvicorn server inside a separate thread.nest_asyncio allows async event loops to run inside Jupyter/Colab.

Start FastAPI server with ngrok, make API accessible via public link.

```
▶ PORT = 8015
  NGROK_AUTH_TOKEN = "31NKQnHnQD38Gz2C5cT4VuCbDUg_7BDr2qrxMqGoKJYqYiQLy"

  ngrok.set_auth_token(NGROK_AUTH_TOKEN)
  public_url = ngrok.connect(PORT).public_url
  print(f" Public API URL: {public_url}")
  public_url = ngrok.connect(PORT).public_url

  nest_asyncio.apply()

  def run_uvicorn():
      uvicorn.run(app, host="0.0.0.0", port=PORT)

  thread = threading.Thread(target=run_uvicorn, daemon=True)
  thread.start()

  time.sleep(3) # wait for server startup
```

```
⇒ Public API URL: https://fa9d1bf59fbd.ngrok-free.app
INFO: Started server process [17928]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8015 (Press CTRL+C to quit)
```



Demo prediction and display graphs inline in Colab . Picks one sample input from test data.

Sends a POST request to your running API to predict fraud.

Displays the prediction response. Fetches and displays the two saved graphs inline in the notebook.

```
sample_features = X_test[0]
sample_features_list = sample_features.tolist()

print("Sample input features:")
print(dict(zip(feature_names, sample_features_list)))

form_data = {feat: str(sample_features_list[i]) for i, feat in
enumerate(feature_names)}
form_data["model"] = "log_reg"

predict_url = f"{public_url}/predict"

print("\nSending prediction request to API...")

response = requests.post(predict_url, data=form_data)
result_json = response.json()

print("Prediction API response:")
display(Markdown(f"**Prediction:**
{result_json.get('prediction')} \n**Model Used:**
{result_json.get('model_used')}"))
```

Display accuracy graph

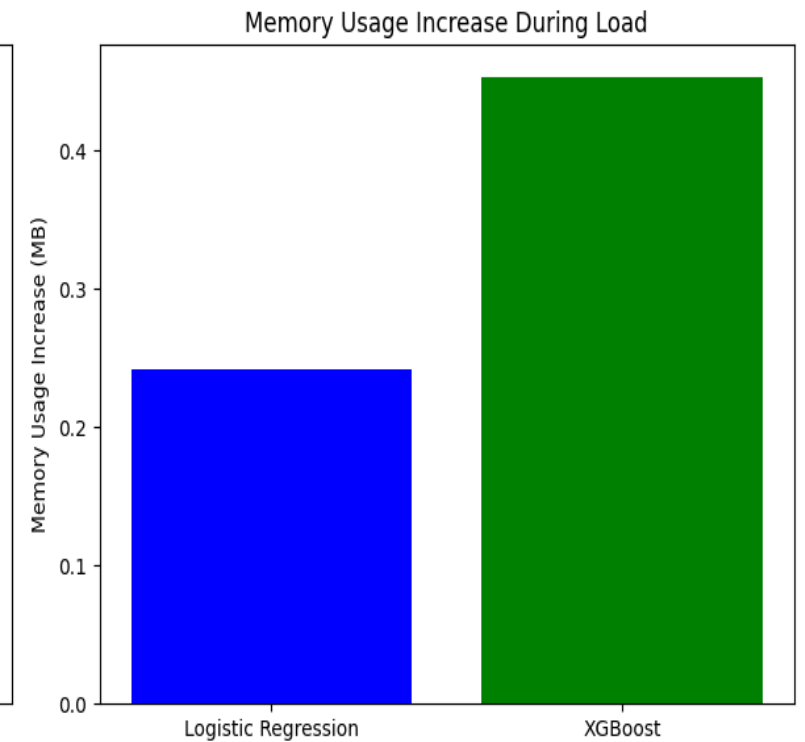
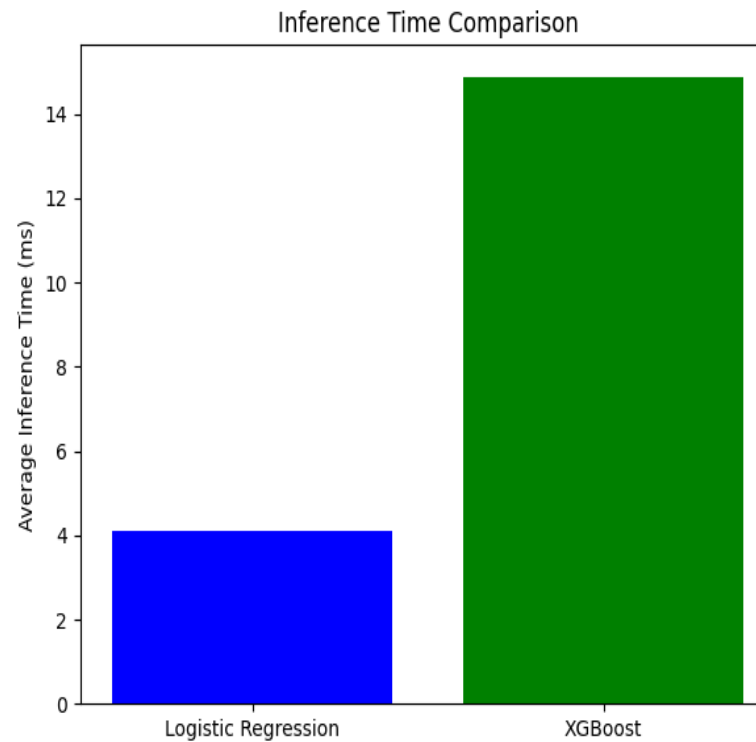
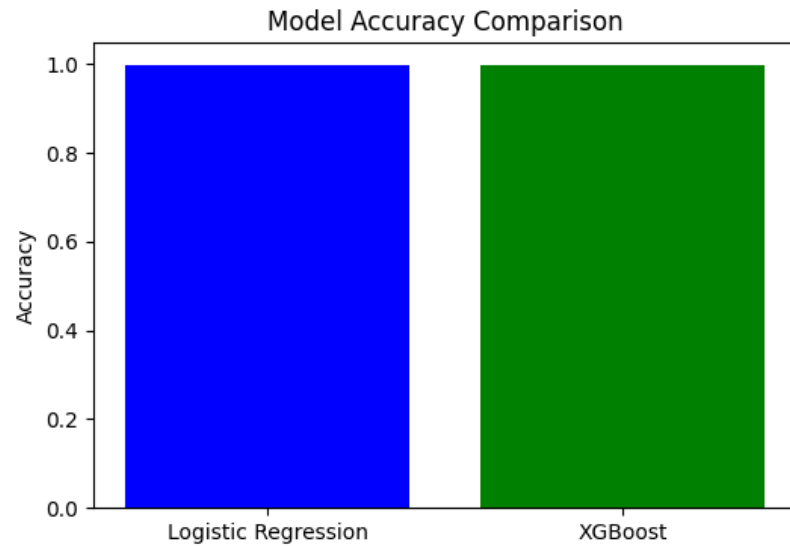
```
graph_url = f"{public_url}/comparison_graph"
img_resp = requests.get(graph_url)
img = Image.open(BytesIO(img_resp.content))
print("\nModel Accuracy Comparison Graph:")
display(img)
```

Display performance graph

```
perf_graph_url = f"{public_url}/performance_graph"
perf_img_resp = requests.get(perf_graph_url)
perf_img =
Image.open(BytesIO(perf_img_resp.content))
print("\nInference Time & Memory Usage Comparison
Graph:")
display(perf_img)
```



Sample input features: {'Time': 0.6839643198001779, 'V1': -1.2954210094135292, 'V2': 1.1450720553408194, 'V3': -0.5543632249055845, 'V4': -0.7788579604135077, 'V5': -1.525617565227062, 'V6': 0.2760785486703751, 'V7': -2.2131301374349697, 'V8': -2.9667279185655646, 'V9': -0.4397718202880524} Sending prediction request to API... INFO: 34.106.110.29:0 - "POST /predict HTTP/1.1" 200 OK Prediction API response: /usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but StandardScaler was fitted with feature names warnings.warn(
Prediction: Legitimate Transaction
Model Used: log_reg
INFO: 34.106.110.29:0 - "GET /comparison_graph HTTP/1.1" 200 OK





Summary of Assignment:

loaded real Credit card fraud detection data, preprocess, train 2 models.

Save models and scaler, plot accuracy graph.

Create API with endpoints for:

Prediction via web form

Serving accuracy and performance graphs

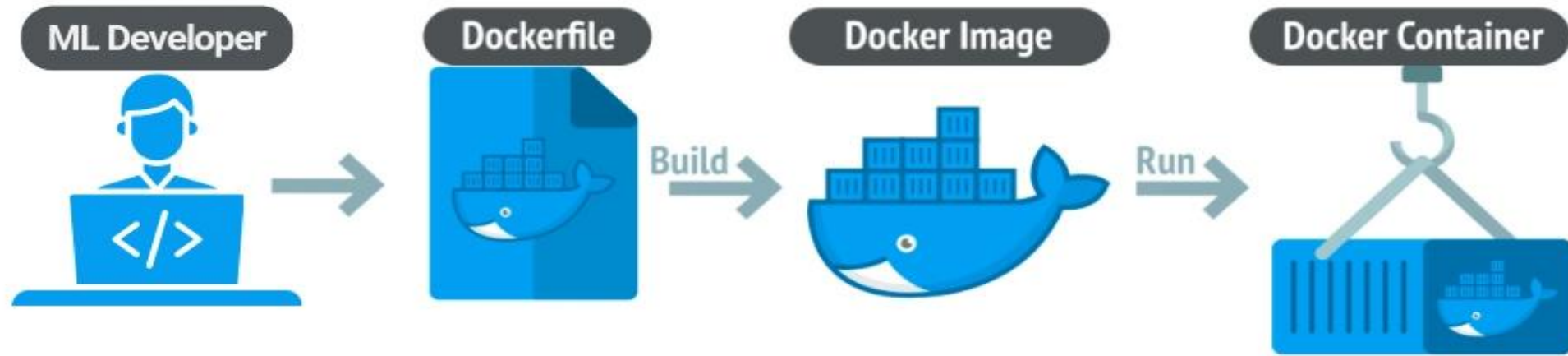
Measure inference time & memory usage under load using concurrency.

Plot and serve performance graph.

Run API publicly using ngrok.

Demo API call & display results and graphs inline in Colab.

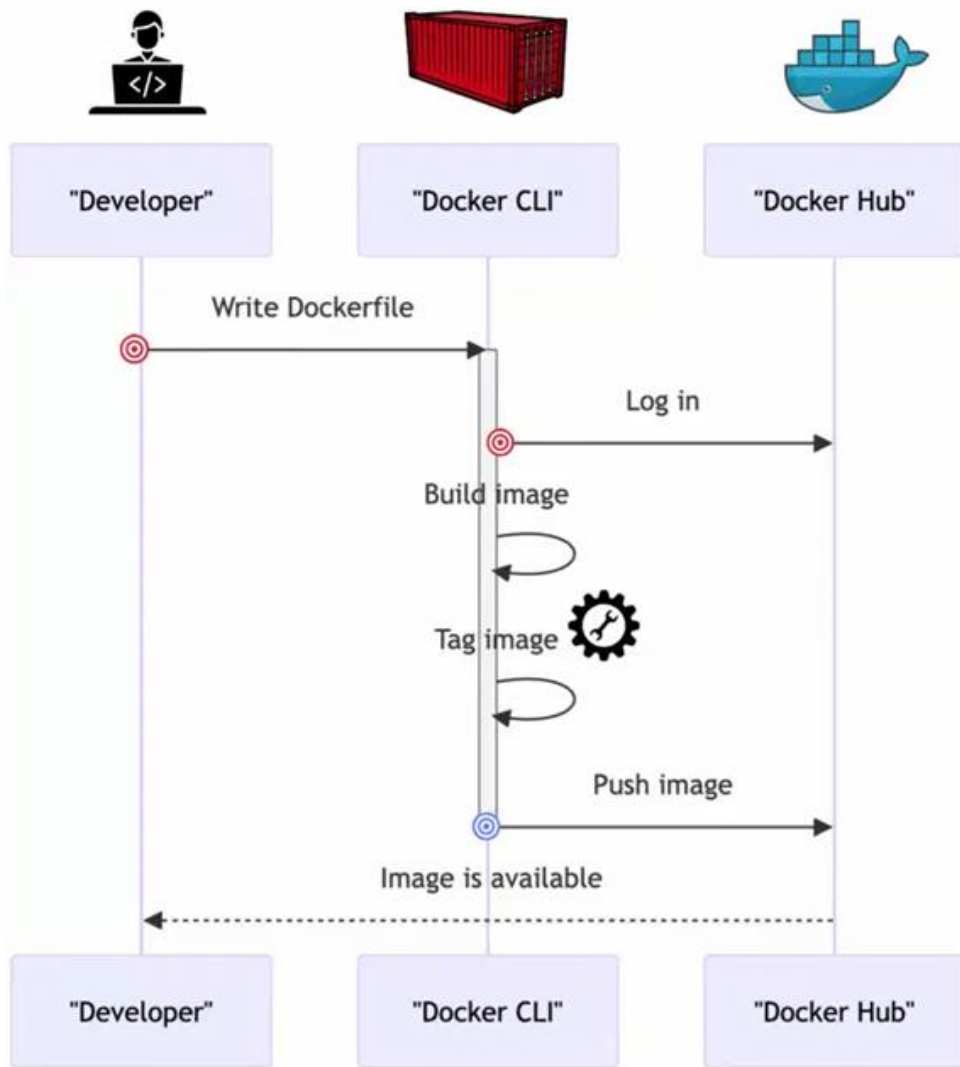
Package and Run ML Models with Docker



Docker is a powerful platform that has **revolutionized the development and distribution of applications by utilizing containerization, a lightweight alternative to full machine virtualization**. Containerization involves encapsulating an application and its environment—dependencies, libraries, and configuration files—into a container, which is a **portable and consistent unit of software**. This approach ensures that the application runs uniformly and consistently across any infrastructure, from a developer's laptop to a high-compute cloud-based server.

Unlike traditional virtual machines that replicate an entire operating system, **Docker containers share the host system's kernel, making them much more efficient, fast to start, and less resource-intensive**. Docker's simple and straightforward syntax hides the complexity often involved in deployment processes, streamlining the workflow and enabling a DevOps approach to lifecycle management of the software development process.

Containerize with Docker



Containerizing ML Project with Docker + Anaconda on Standalone Machine Step:1

```
# Local project path
project_path = r"F:\TEACHING NOTES\AMLIS\AMLIS Lab\fraud_detection_project"

# Docker settings
python_version = "3.11.5"      # Python version for Docker
fastapi_port = 8037            # Port to expose
docker_image_name = "fraud-detection" # Name of the Docker image
```



Containerize with Docker

Step-2: Containerizing ML Project with Docker + Anaconda on Standalone Machine

```
dockerfile_content = f"""
# Use official Python {python_version} base image
FROM python:{python_version}

# Set working directory inside the container
WORKDIR /app

# Copy all project files to container
COPY . /app

# Install required packages
RUN pip install --no-cache-dir fastapi uvicorn scikit-learn xgboost pandas matplotlib joblib gdown python-multipart

# Expose port for FastAPI
EXPOSE {fastapi_port}

# Command to run the FastAPI app (assuming app.py exists with FastAPI instance 'app')
CMD ["uvicorn", "app:app", "--host", "127.0.0.1", "--port", "{fastapi_port}"]
"""
```



Containerize with Docker

Step-3: Containerizing ML Project with Docker + Anaconda on Standalone Machine

```
import os

# Dockerfile path
dockerfile_path = os.path.join(project_path, "Dockerfile")

# Write Dockerfile content
with open(dockerfile_path, "w") as f:
    f.write(dockerfile_content)

print(f"Dockerfile created successfully at: {dockerfile_path}")
```

Dockerfile created successfully at: F:\TEACHING NOTES\AMLIS\AMLIS Lab\fraud_detection_project\Dockerfile



Containerize with Docker

Step-4: Containerizing ML Project with Docker + Anaconda on Standalone Machine

```
# Read and display Dockerfile content
with open(dockerfile_path, "r") as f:
    print(f.read())
```

```
# Use official Python 3.11.5 base image
FROM python:3.11.5
```

```
# Set working directory inside the container
WORKDIR /app
```

```
# Copy all project files to container
COPY . /app
```

```
# Install required packages
RUN pip install --no-cache-dir fastapi uvicorn scikit-learn xgboost pandas matplotlib joblib gdown python-multipart
```

```
# Expose port for FastAPI
EXPOSE 8037
```

```
# Command to run the FastAPI app (assuming app.py exists with FastAPI instance 'app')
CMD ["uvicorn", "app:app", "--host", "127.0.0.1", "--port", "8037"]
```



•Containerize with Docker

Step-5:Containerizing ML Project with Docker + Anaconda on Standalone Machine

Build the Docker image

Open a terminal/PowerShell, change directory to your project folder:

```
bash
```

```
cd "F:\TEACHING NOTES\AMLIS\AMLIS Lab\fraud_detection_project"
```

Run:

```
bash
```

```
docker build -t fraud-detection-app .
```

This will build the Docker image and install your Anaconda environment plus copy your code.



Containerize with Docker

Step-6: Build The Docker Image of ML Project with Docker + Anaconda on Standalone Machine

```
(base) F:\TEACHING NOTES\AMLIS\AMLIS Lab\fraud_detection_project>docker build -t fraud-detection-app .
[+] Building 308.6s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 566B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.11.5    2.7s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [internal] load build context                                   0.1s
=> => transferring context: 261.58kB                                0.0s
=> [1/4] FROM docker.io/library/python:3.11.5@sha256:2e376990a11f1c1e03796d08db0e99c36eadb4bb6491372b227f1e53c34 0.0s
=> CACHED [2/4] WORKDIR /app                                       0.0s
=> [3/4] COPY . /app                                              1.1s
=> [4/4] RUN pip install --no-cache-dir fastapi uvicorn scikit-learn xgboost pandas matplotlib joblib gdown py 288.5s
=> exporting to image                                              16.0s
=> => exporting layers                                              15.9s
=> => writing image sha256:28aa1f33484a6e15881bf61829ac3ebec1bd03fa91a5124cc40ba24deafd7760 0.0s
=> => naming to docker.io/library/fraud-detection-app             0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/jmljw1pnw17ziodospaw3f78t
```



Containerize with Docker

Step-7:Share Docker image , we can push this image to Docker Hub or private registry:

Select Administrator: Anaconda Prompt - CMD ["conda", "run", "--no-capture-output", "-n", "your_env_name", "python", "your_main_script.py"]

```
(base) F:\TEACHING NOTES\AMLIS\AMLIS Lab\fraud_detection_project>docker login
Authenticating with existing credentials... [Username: tbhaskar22]
```

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded

```
(base) F:\TEACHING NOTES\AMLIS\AMLIS Lab\fraud_detection_project>docker tag fraud-detection-app tbhaskar22/fraud-detection-app:latest
```

```
(base) F:\TEACHING NOTES\AMLIS\AMLIS Lab\fraud_detection_project>docker push tbhaskar22/fraud-detection-app:latest
The push refers to repository [docker.io/tbhaskar22/fraud-detection-app]
```

56dee65ad9e1: Pushed

b4665c40fc17: Pushed

f23d19fb1ef3: Pushed

db22e0d1d36b: Mounted from library/python

0d3f1aea6da4: Mounted from library/python

78dd9ecf8a6d: Mounted from library/python

c26432533a6a: Mounted from library/python

01d6cdeac539: Mounted from library/python

a981dddd4c65: Mounted from library/python

f6589095d5b5: Mounted from library/python

7c85cfa30cb1: Mounted from library/python

latest: digest: sha256:f99389c3fca3cb4916ae4f7be8987e7cc20d9323969fe4de3e791541300c38fe size: 2638

```
(base) F:\TEACHING NOTES\AMLIS\AMLIS Lab\fraud_detection_project>_
```



Containerize with Docker

Step-8: Docker Image of ML Project is available in Docker Hub or private registry:

tbhaskar22

Ask Gordon

BETA

Containers

Images

Volumes

Builds

Models

BETA

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

[Give feedback](#)

View and manage your local and Docker Hub images.

[Learn more](#)

Local

My Hub

3.31 GB / 0 Bytes in use

2 images

Last refresh: 7 hours ago

Search

	Name	Tag	Image ID	Created	Size	Actions
☐	<none>	<none>	1f67b2fa6643	2 hours ago	2.23 GB	
☐	fraud-detection-app	latest	28aa1f33484a	1 hour ago	2.23 GB	
☐	tbhaskar22/fraud-detection-app	latest	28aa1f33484a	1 hour ago	2.23 GB	

Dr.T.Bhaskar,Dept. of Computer Engineering, Sanjivani CoE, Kopargaon

78



Containerize with Docker

Step-9: Pull and use the Docker image on any machine

Anyone with access (you or your team) can download (pull) the image and run it:

`docker run -it -p 8037:8000 tbhaskar22/fraud-detection-app:latest`

- That means: forward your local port 8037 to container port 8000.
- After running, visit <http://127.0.0.1:8037> in your browser.

```
Administrator: Anaconda Prompt - CMD ["conda", "run", "--no-capture-output", "-n", "your_env_name", "python", "your_main_script.py"] - docker run -it -p 8037:8000 tbhaskar22/fraud-detection-app:latest

(base) F:\TEACHING NOTES\AMLIS\AMLIS Lab\fraud_detection_project>docker run -it -p 8037:8000 tbhaskar22/fraud-detection-app:latest
/usr/local/lib/python3.11/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator LogisticRegression from version 1.3.0 when using version 1.7.1. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
/usr/local/lib/python3.11/site-packages/sklearn/base.py:442: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from version 1.3.0 when using version 1.7.1. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8037 (Press CTRL+C to quit)
```



Containerize with Docker

Step-10: After Pulling the Docker image on any machine

We can Run Credit Card Fraud Detection on local host: <http://127.0.0.1:8037> in browser

Output of Log Reg

← → ↻ ⓘ 127.0.0.1:8037

📦 | 🌐 Gmail 🔄 Course List 🏰 Paper Cheking 🏠 Home | Dr.T.Bhaskar... 🔄 Adk

Fraud Detection

feat0:	1
feat1:	2
feat2:	3
feat3:	4
feat4:	5
feat5:	6
feat6:	7
feat7:	8
feat8:	9
feat9:	10

LogReg ▾

Submit

← → ↻ ⓘ 127.0.0.1:8037/predict

📦 | 🌐 Gmail 🔄 Course List 🏰 Paper Cheking 🏠 Home | Dr.T.Bhaskar...

Pretty-print ☐

```
{"prediction":"Not Fraud","model":"log_reg"}
```

Output of XGB

← → ↻ ⓘ 127.0.0.1:8037/predict

📦 | 🌐 Gmail 🔄 Course List 🏰 Paper Cheking 🏠 Home | Dr.T.Bhaskar...

Pretty-print ☐

```
{"prediction":"Not Fraud","model":"xgb"}
```

<

IN USE

28aa1f33484a

CREATED
2 hours ago

SIZE
2.23 GB

Recommended fixes ▾

Run

-

```
3 apt-get update && apt-get install -y --no-install-recommen... 176.65 MB
```

```
4 set -ex; apt-get update; apt-get install -y --no-install-recom... 587.12 MB
```

```
5 ENV PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/... 0 B
```

```
6      ENV LANG=C.UTF-8                                0 B
```

```
7 RUN /bin/sh -c set -eux; apt-get update; apt-get install -y -... 18.64 MB
```

```
8  ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8... 0 B
```

```
9 ENV PYTHON_VERSION=3.11.5 0 B
```

10	RUN /bin/sh -c set -eux; wget -O python.tar.xz "https://ww...	50.08 MB
----	---	----------

```
11 RUN /bin/sh -c set -eux; for src in idle3 pydoc3 python3 p... 32 B
```

```
12 ENV PYTHON_PIP_VERSION=23.2.1 0 B
```

```
13 ENV PYTHON_SETUPTOOLS_VERSION=65.5.1 0 B
```

```
14 ENV PYTHON_GET_PIP_URL=https://github.com/pypa/get-pip.py 0 B
```

This image has not been analyzed

You can use Docker Scout to analyze local images and list its vulnerabilities.

Start analysis

[Enable background indexing in Settings](#) so your results are always ready.



PERSONAL



Ctrl+K







T



tbhaskar22



Ask Gordon BETA



Containers



Images



Volumes



Builds



Models BETA



MCP Toolkit BETA



Docker Hub



Docker Scout



Extensions

Images / tbhaskar22/fraud-detection-app:latest

<

tbhaskar22/fraud-detection-app:latest

IN USE

28aa1f33484a

CREATED

2 hours ago

SIZE

2.23 GB

Recommended fixes

▼

Run

▼

Layers (23)

0	ADD file:ce04d6a354feaf93795269c859f36667fce9efda...	116.49 MB
1	CMD ["bash"]	0 B
2	set -eux; apt-get update; apt-get install -y --no-install-reco...	48.4 MB
3	apt-get update && apt-get install -y --no-install-recommen...	176.65 MB
4	set -ex; apt-get update; apt-get install -y --no-install-recom...	587.12 MB
5	ENV PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/...	0 B
6	ENV LANG=C.UTF-8	0 B
7	RUN /bin/sh -c set -eux; apt-get update; apt-get install -y -...	18.64 MB
8	ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8...	0 B
9	ENV PYTHON_VERSION=3.11.5	0 B

Vulnerabilities

Packages

Analyzed by



This image has not been analyzed

You can use Docker Scout to analyze local images and list its vulnerabilities.

Start analysis

[Enable background indexing in Settings](#) so your results are always ready.

Dr.T.Bhaskar,Dept. of Computer Engineering, Sanjivani CoE, Kopargaon

82





Advantages of Containerization Using Docker

Docker allows you to **package your application with everything it needs**—code, dependencies, libraries, and runtime—into a **single container**.

Key Advantages:

1.Consistency Across Environments

1. "It works on my machine" problem disappears.
2. Your fraud detection API behaves the same on your laptop, server, or cloud.

2.Lightweight & Fast

1. Containers share the host OS kernel, unlike virtual machines.
2. Start/stop times are faster, which is crucial for **real-time applications**.

3.Isolation

1. Each container runs independently, avoiding conflicts between different projects or libraries.
2. Example: You can run **Logistic Regression** container alongside **XGBoost** container without version conflicts.

4.Portability

1. Docker containers run anywhere: Windows, Linux, Mac, or any cloud platform.
2. You don't have to manually install Python, libraries, or dependencies on every machine.

5.Version Control & Rollback

1. You can version your Docker images.
2. Rollback to a previous working version if a new update breaks your app.

6.Scalability & Microservices Friendly

1. Easily replicate containers to handle more traffic.
2. Supports modern architecture: divide your system into **microservices** (e.g., fraud detection service, user management service).

7.Resource Efficiency

1. Containers are smaller than virtual machines.
2. Run more containers on the same hardware.



Deploy on cloud (AWS, Azure)

AWS ECS (Fargate)

1. Push Docker image to **Amazon ECR**.
2. Create an **ECS Cluster** → **Fargate Task** → Deploy your container.
3. Open port 8000 in **security group**.
4. Access API using public endpoint.

Azure App Service

1. Push Docker image to **Azure Container Registry (ACR)**.
2. Create **Web App for Containers**, select your image.
3. API available via Azure-provided URL.



Advantages of Cloud Deployment (AWS & Azure)

Deploying on cloud platforms adds **high availability, scalability, and security** to your system.

Key Advantages:

1.Global Accessibility

1. Your API can be accessed anywhere via a public URL.
2. Example: Banks or financial services can call your fraud detection API from multiple regions.

2.Scalability & Auto-scaling

1. Cloud platforms can automatically spin up more instances if requests increase.
2. Ensures your real-time API doesn't slow down under heavy load.

3.Managed Infrastructure

1. No need to manually manage servers, networking, or storage.
2. AWS ECS, Azure App Service, and Fargate handle underlying infrastructure.

4.High Availability & Fault Tolerance

1. Cloud platforms distribute containers across multiple servers and regions.
2. If one server fails, another takes over, preventing downtime.

5.Monitoring & Logging

1. AWS CloudWatch, Azure Monitor, and Application Insights track performance, errors, CPU/memory usage.
2. Helps detect slow inference or memory bottlenecks in your fraud detection API.

6.Security

1. Role-based access control, firewall rules, SSL, and identity management.
2. Protects sensitive financial data processed by the API.

7.Cost Efficiency

1. Pay only for resources used.
2. Auto-scaling ensures you're not paying for idle servers.



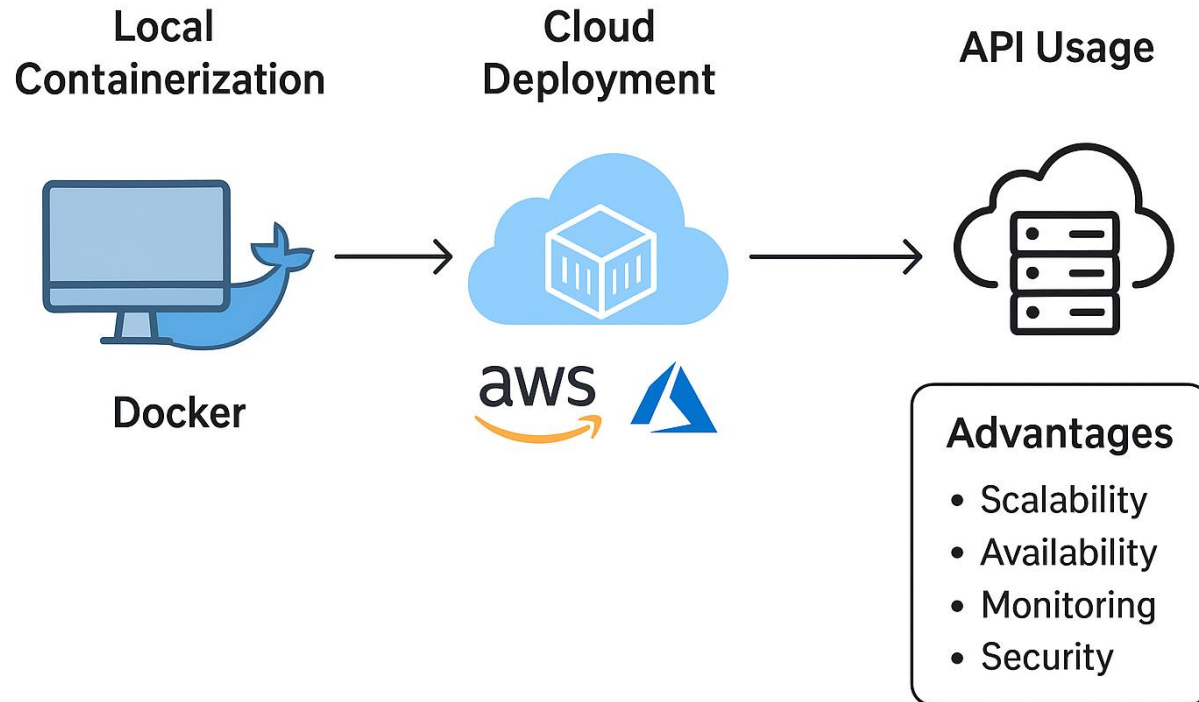
Practical Benefits for Your Fraud Detection System

Aspect	Without Docker/Cloud	With Docker & Cloud
Deployment	Manual, error-prone	One image, runs anywhere
Scalability	Hard to scale	Auto-scale containers as requests grow
Consistency	May fail on different machines	Same behavior everywhere
Performance Monitoring	Hard to track	Cloud monitoring tools
Security	Limited	Managed security and access control
Maintenance	Requires manual updates	Update image and redeploy easily

Summary:

- **Docker** → Makes your application **portable, consistent, lightweight, and easy to scale**.
- **Cloud Platforms (AWS/Azure)** → Provide **availability, scalability, monitoring, and security**, allowing your real-time fraud detection system to handle **millions of transactions reliably**.

Practical Benefits for Your Fraud Detection System



Summary:

• **Docker** → Makes your application **portable, consistent, lightweight, and easy to scale.**

• **Cloud Platforms (AWS/Azure)** → Provide **availability, scalability, monitoring, and security**, allowing your real-time fraud detection system to handle **millions of transactions reliably.**



Practical Benefits for Your Fraud Detection System

Aspect	Standalone Anaconda	NGROK + Anaconda (Local)	Docker (Local / Server)	Docker + Cloud
1. Deployment	Simple on one machine; ideal for dev or research use.	Instant exposure to public via tunnel, fast for testing/demos.	Containerized, reproducible deployments; works across dev, staging, prod.	Automated CI/CD; cloud-native deployments with orchestration (e.g. Kubernetes, ECS).
2. Scalability	Not scalable; limited by local machine.	Not scalable; single-user demo.	Scalable with orchestration (Docker Swarm/K8s); suitable for small-medium workloads.	Highly scalable; auto-scaling, load balancing, global distribution for real-time detection.
3. Consistency	Medium: Conda ensures env control but platform-dependent.	Same as Anaconda; depends on local setup.	High: Containerization ensures identical behavior across machines.	Very High: Immutable infra via containers + IaC ensures full consistency.
4. Performance Monitoring	Manual logging/print statements; no built-in monitoring.	Basic request logs via NGROK; limited visibility.	Integrates with Prometheus, Grafana, ELK Stack, etc. for metrics/logs.	Full observability with cloud-native tools (CloudWatch, Stackdriver, etc.).
5. Security	Low: No isolation, user-dependent security.	Low-Medium: Exposed tunnel can be risky; can add auth on NGROK Pro.	Medium-High: Containers offer isolation; can secure networks and volumes.	Enterprise-grade: IAM, VPCs, encrypted storage, secret managers, firewalls.
6. Maintenance	Manual updates, package conflicts can arise.	Same as Anaconda; requires user vigilance.	Easy versioning and rollbacks; maintenance via image rebuilds.	Automated deployments, patching, rolling updates, backups; DevOps-ready.
7. Cost	Free (local resources only).	Free for basic tunnel; NGROK Pro adds features.	Low for small servers; self-hosted infra costs.	Pay-as-you-go; scalable costs but efficient for high load systems.
8. Use Case Fit	R&D, prototyping, algorithm tuning.	Fast prototyping, client demos, proof of concept.	Testing environments, small production setups, controlled enterprise setups.	Full-scale enterprise fraud systems with high availability & fault tolerance.
9. Setup Complexity	Low. Install Anaconda, start coding.	Very Low. Install NGROK, run local service.	Medium. Requires Docker knowledge and some system integration.	High. Requires DevOps setup, cloud account, orchestration knowledge.
10. Collaboration	Limited; needs export of notebooks or files.	Easy demo sharing via public URL.	Good; share images, use docker-compose or registries.	Excellent; multi-team workflows, cloud repos, automated pipelines.



Suggested Readings

Text Books:

1. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron - Covers Unit I & II: Foundational ML concepts, supervised & unsupervised learning, and data preprocessing.
2. Pattern Recognition and Machine Learning" by Christopher M. Bishop - Supports Unit I & IV: Deep dive into mathematical foundations and evaluation techniques.
3. Machine Learning Engineering" by Andriy Burkov - Excellent for Unit III: Practical model deployment, CI/CD, and ML systems in industry
4. Deep Learning with Python" by François Chollet - Addresses Unit V: Includes foundation models, generative AI, and on-device deep learning.

Reference Books:

1. Introduction to Statistical Learning" by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani - Relevant for Units I & II: Statistical methods, linear models, and feature engineering.
2. Designing Machine Learning Systems with Python" by David Julian - Supports Unit III: Implementation, pipelines, and deployment using industry tools
3. Ethics of Artificial Intelligence and Robotics" (Stanford Encyclopedia of Philosophy) - Unit IV: Ethical, societal, and legal implications of ML and AI.
4. Graph Representation Learning" by William L. Hamilton - Unit V: In-depth reference on Graph Neural Networks and their applications.